



Model Training

1.1 Import Data and Required Packages

Importing Pandas, Numpy, Matplotlib, Seaborn and Warnings Library.

```
In [6]: # Basic Import
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Modelling
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, AdaBoostRegressor
from sklearn.svm import SVR
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
from sklearn.model_selection import RandomizedSearchCV
from catboost import CatBoostRegressor
from xgboost import XGBRegressor
import warnings
```

Import the CSV Data as Pandas DataFrame

```
In [8]: df = pd.read_csv('data/stud.csv')
```

Show Top 5 Records

```
In [9]: df.head()
```

```
Out[9]:
```

	gender	race_ethnicity	parental_level_of_education	lunch	test_preparation
0	female	group B	bachelor's degree	standard	
1	female	group C	some college	standard	c
2	female	group B	master's degree	standard	
3	male	group A	associate's degree	free/ reduced	
4	male	group C	some college	standard	

Preparing X and Y variables

```
In [10]: X = df.drop(columns=['math_score'],axis=1)
```

```
In [11]: X.head()
```

```
Out[11]:
```

	gender	race_ethnicity	parental_level_of_education	lunch	test_preparation
0	female	group B	bachelor's degree	standard	
1	female	group C	some college	standard	c
2	female	group B	master's degree	standard	
3	male	group A	associate's degree	free/reduced	
4	male	group C	some college	standard	

```
In [14]: print("Categories in 'gender' variable:      ",end=" ")
print(df['gender'].unique())

print("Categories in 'race_ethnicity' variable:  ",end=" ")
print(df['race_ethnicity'].unique())

print("Categories in 'parental level of education' variable:",end=" ")
print(df['parental_level_of_education'].unique())

print("Categories in 'lunch' variable:          ",end=" ")
print(df['lunch'].unique())

print("Categories in 'test preparation course' variable:      ",end=" ")
print(df['test_preparation_course'].unique())
```

Categories in 'gender' variable: ['female' 'male']
Categories in 'race_ethnicity' variable: ['group B' 'group C' 'group A' 'group D' 'group E']
Categories in 'parental level of education' variable: ["bachelor's degree" 'some college' "master's degree" "associate's degree" 'high school' 'some high school']
Categories in 'lunch' variable: ['standard' 'free/reduced']
Categories in 'test preparation course' variable: ['none' 'completed']

```
In [12]: y = df['math_score']
```

```
In [13]: y
```

```
Out[13]: 0      72
1      69
2      90
3      47
4      76
..
995    88
996    62
997    59
998    68
999    77
Name: math_score, Length: 1000, dtype: int64
```

```
In [15]: # Create Column Transformer with 3 types of transformers
num_features = X.select_dtypes(exclude="object").columns
cat_features = X.select_dtypes(include="object").columns

from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer

numeric_transformer = StandardScaler()
oh_transformer = OneHotEncoder()

preprocessor = ColumnTransformer(
    [
        ("OneHotEncoder", oh_transformer, cat_features),
        ("StandardScaler", numeric_transformer, num_features),
    ]
)
```

```
In [16]: X = preprocessor.fit_transform(X)
```

```
In [18]: X.shape
```

```
Out[18]: (1000, 19)
```

```
In [21]: # separate dataset into train and test
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_s
X_train.shape, X_test.shape
```

```
Out[21]: ((800, 19), (200, 19))
```

Create an Evaluate Function to give all metrics after model Training

```
In [22]: def evaluate_model(true, predicted):
    mae = mean_absolute_error(true, predicted)
    mse = mean_squared_error(true, predicted)
    rmse = np.sqrt(mean_squared_error(true, predicted))
    r2_square = r2_score(true, predicted)
    return mae, rmse, r2_square
```

```
In [23]: models = {
    "Linear Regression": LinearRegression(),
    "Lasso": Lasso(),
    "Ridge": Ridge(),
    "K-Neighbors Regressor": KNeighborsRegressor(),
    "Decision Tree": DecisionTreeRegressor(),
    "Random Forest Regressor": RandomForestRegressor(),
    "XGBRegressor": XGBRegressor(),
    "CatBoosting Regressor": CatBoostRegressor(verbose=False),
    "AdaBoost Regressor": AdaBoostRegressor()
}
model_list = []
```

```

r2_list =[]

for i in range(len(list(models))):
    model = list(models.values())[i]
    model.fit(X_train, y_train) # Train model

    # Make predictions
    y_train_pred = model.predict(X_train)
    y_test_pred = model.predict(X_test)

    # Evaluate Train and Test dataset
    model_train_mae , model_train_rmse, model_train_r2 = evaluate_model(y_train, y_train_pred)
    model_test_mae , model_test_rmse, model_test_r2 = evaluate_model(y_test, y_test_pred)

    print(list(models.keys())[i])
    model_list.append(list(models.keys())[i])

    print('Model performance for Training set')
    print("- Root Mean Squared Error: {:.4f}".format(model_train_rmse))
    print("- Mean Absolute Error: {:.4f}".format(model_train_mae))
    print("- R2 Score: {:.4f}".format(model_train_r2))

    print('-----')

    print('Model performance for Test set')
    print("- Root Mean Squared Error: {:.4f}".format(model_test_rmse))
    print("- Mean Absolute Error: {:.4f}".format(model_test_mae))
    print("- R2 Score: {:.4f}".format(model_test_r2))
    r2_list.append(model_test_r2)

print('='*35)
print('\n')

```

Linear Regression

Model performance for Training set

- Root Mean Squared Error: 5.3243
- Mean Absolute Error: 4.2671
- R2 Score: 0.8743

Model performance for Test set

- Root Mean Squared Error: 5.3960
- Mean Absolute Error: 4.2158
- R2 Score: 0.8803

=====

Lasso

Model performance for Training set

- Root Mean Squared Error: 6.5938
- Mean Absolute Error: 5.2063
- R2 Score: 0.8071

Model performance for Test set

- Root Mean Squared Error: 6.5197
- Mean Absolute Error: 5.1579
- R2 Score: 0.8253

=====

Ridge

Model performance for Training set

- Root Mean Squared Error: 5.3233
- Mean Absolute Error: 4.2650
- R2 Score: 0.8743

Model performance for Test set

- Root Mean Squared Error: 5.3904
- Mean Absolute Error: 4.2111
- R2 Score: 0.8806

=====

K-Neighbors Regressor

Model performance for Training set

- Root Mean Squared Error: 5.7077
- Mean Absolute Error: 4.5167
- R2 Score: 0.8555

Model performance for Test set

- Root Mean Squared Error: 7.2530
- Mean Absolute Error: 5.6210
- R2 Score: 0.7838

=====

Decision Tree

Model performance for Training set

- Root Mean Squared Error: 0.2795
- Mean Absolute Error: 0.0187
- R2 Score: 0.9997

Model performance for Test set

- Root Mean Squared Error: 7.6371
- Mean Absolute Error: 6.0250
- R2 Score: 0.7603

=====

Random Forest Regressor

Model performance for Training set

- Root Mean Squared Error: 2.2851
- Mean Absolute Error: 1.8253
- R2 Score: 0.9768

Model performance for Test set

- Root Mean Squared Error: 6.0959
- Mean Absolute Error: 4.7194
- R2 Score: 0.8473

=====

XGBRegressor

Model performance for Training set

- Root Mean Squared Error: 0.9087
- Mean Absolute Error: 0.6148
- R2 Score: 0.9963

Model performance for Test set

- Root Mean Squared Error: 6.5889
- Mean Absolute Error: 5.0844
- R2 Score: 0.8216

=====

CatBoosting Regressor

Model performance for Training set

- Root Mean Squared Error: 3.0427
- Mean Absolute Error: 2.4054
- R2 Score: 0.9589

Model performance for Test set

- Root Mean Squared Error: 6.0086
- Mean Absolute Error: 4.6125
- R2 Score: 0.8516

=====

AdaBoost Regressor

Model performance for Training set

- Root Mean Squared Error: 5.7843
- Mean Absolute Error: 4.7564

- R2 Score: 0.8516

Model performance for Test set

- Root Mean Squared Error: 6.0447

- Mean Absolute Error: 4.6813

- R2 Score: 0.8498
=====

Results

```
In [24]: pd.DataFrame(list(zip(model_list, r2_list)), columns=['Model Name', 'R2_Score'])
```

```
Out[24]:
```

	Model Name	R2_Score
2	Ridge	0.880593
0	Linear Regression	0.880345
7	CatBoosting Regressor	0.851632
8	AdaBoost Regressor	0.849847
5	Random Forest Regressor	0.847291
1	Lasso	0.825320
6	XGBRegressor	0.821589
3	K-Neighbors Regressor	0.783813
4	Decision Tree	0.760313

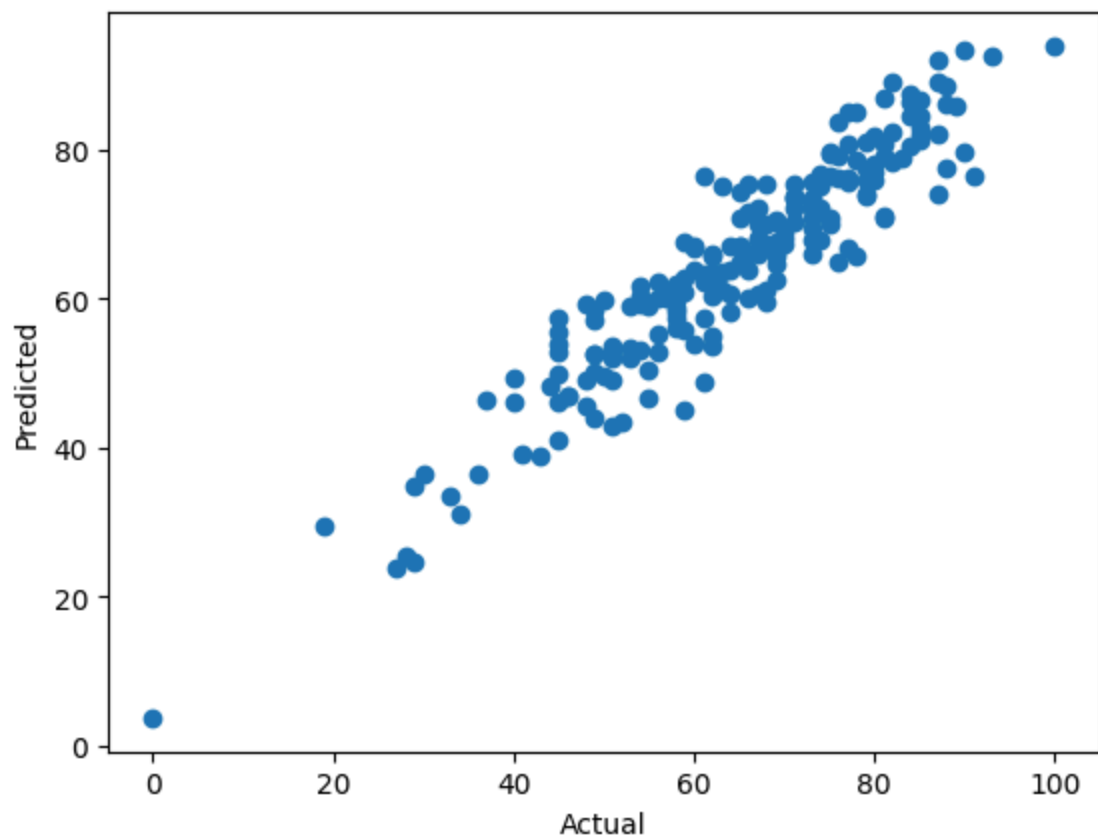
Linear Regression

```
In [25]: lin_model = LinearRegression(fit_intercept=True)
lin_model = lin_model.fit(X_train, y_train)
y_pred = lin_model.predict(X_test)
score = r2_score(y_test, y_pred)*100
print(" Accuracy of the model is %.2f" %score)
```

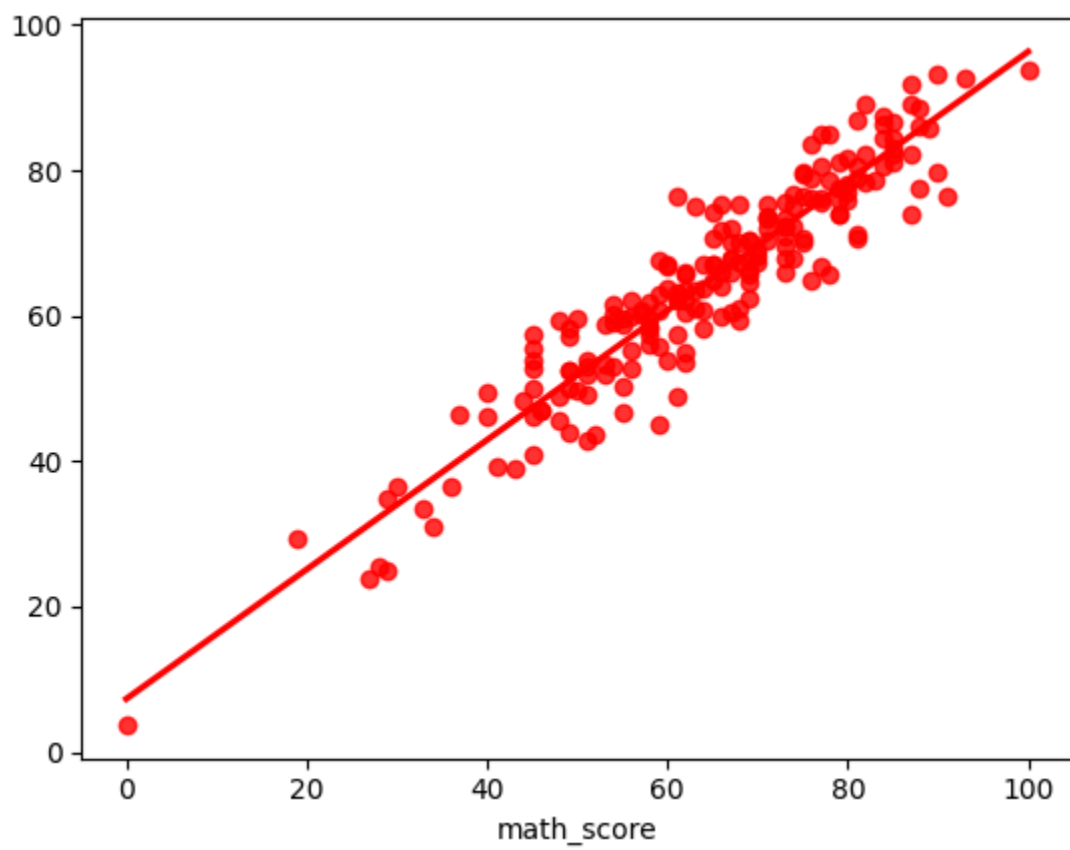
Accuracy of the model is 88.03

Plot y_pred and y_test

```
In [26]: plt.scatter(y_test,y_pred);
plt.xlabel('Actual');
plt.ylabel('Predicted');
```



```
In [27]: sns.regplot(x=y_test,y=y_pred,ci=None,color='red');
```



Difference between Actual and Predicted Values

```
In [28]: pred_df=pd.DataFrame({'Actual Value':y_test,'Predicted Value':y_pred,'Difference':y_test-y_pred})
```

```
Out[28]:
```

	Actual Value	Predicted Value	Difference
521	91	76.507812	14.492188
737	53	58.953125	-5.953125
740	80	76.960938	3.039062
660	74	76.757812	-2.757812
411	84	87.539062	-3.539062
...	...	...	...
408	52	43.546875	8.453125
332	62	62.031250	-0.031250
208	74	67.976562	6.023438
613	65	67.132812	-2.132812
78	61	62.492188	-1.492188

200 rows × 3 columns

```
In [ ]:
```